ADVANCED CRYPTOGRAPHY | CSPE-71

MINI PROJECT

SOLIDITY SMART CONTRACT MUTATION TESTING TOOL

Injecting Security-Relevant Faults to Evaluate Test-Suite Effectiveness

SUBMITTED BY

Garvv Tatia

Roll No.: 106123040

SUBJECT DETAILS

Subject: Advanced Cryptography

Subject Code: CSPE-71

PROJECT FOCUS

Unchecked Arithmetic • Reentrancy • Timestamp Reliance

Submitted on 28th October 2026

Contents

01 Abstract

02 1. Introduction

03 2. Scope and Assumptions

04 3. System Architecture

05 4. Mutation Operator Design

06 5. Execution Workflow

07 6. Mutation Score and Metrics

08 7. Experimental Setup and Results

09 8. Survivor Analysis and Test Hardening

10 9. Verification and Quality Assurance

11 10. Deliverables

12 11. Limitations and Threats to Validity

13 12. Conclusion

14 Appendix A. Example Commands

15 Appendix B. References and Further Reading

Appendices include example commands and references.

Abstract

Smart contracts often manage valuable digital assets and execute without a central intermediary. Although automated tests are widely used to validate smart contracts, conventional code-coverage metrics primarily measure which code was executed; they do not establish whether the test suite can detect incorrect or security-relevant behavior. This project develops a local, AST-guided mutation-testing framework for Solidity contracts. The framework applies three classes of controlled mutations—unchecked arithmetic, reentrancy-related statement reordering, and timestamp-dependent boundary changes—then compiles each mutant, runs the target project's Foundry test suite, classifies outcomes, calculates mutation scores, and produces JSON and standalone HTML reports.

The supplied implementation report records 24 passing pytest tests and 9 passing Foundry tests for the sample project. It also records a mutation campaign of 11 mutants, with 3 killed and 8 surviving, yielding a reported overall mutation score of 27.27%. These results suggest that the sample test suite detects the selected reentrancy and some timestamp mutations, while arithmetic-boundary mutations remain undetected. Mutation scores are interpreted as measures of sensitivity to the selected fault model, not as direct measures of contract security.

1. Introduction

1.1 Background

Solidity is the primary programming language used to implement many Ethereum smart contracts. Smart contracts can hold assets, enforce permissions, manage deadlines, and implement financial logic. Defects in these contracts may lead to unauthorized state changes, loss of funds, or unexpected transaction behavior. Testing is therefore a critical part of smart contract development.

Traditional testing practices commonly report statement, branch, or function coverage. Coverage indicates whether a test executed a portion of the code, but execution alone does not prove that assertions would detect a faulty result. A test may call a withdrawal function and achieve coverage without verifying that balances remain correct under adversarial execution.

1.2 Problem statement

The project addresses the gap between code execution and test effectiveness. It introduces controlled, security-relevant faults into copies of Solidity contracts and measures whether the existing tests detect those faults. A fault that causes an attributable test failure is considered killed; a fault that changes behavior but is not detected may survive.

1.3 Research question

> ***How effectively does an existing Solidity smart contract test suite detect deliberately introduced, security-relevant faults?***

1.4 Objectives

- Develop a modular mutation-testing framework for Solidity projects.

- Implement operators for unchecked arithmetic, reentrancy-related statement ordering, and timestamp-dependent logic.
- Compile each mutant and run the existing test suite in an isolated workspace.
- Classify mutant outcomes transparently and calculate overall and operator-specific mutation scores.
- Generate machine-readable JSON results and a standalone HTML report.
- Use survivor analysis to identify missing tests and recommend targeted test improvements.

2. Scope and Assumptions

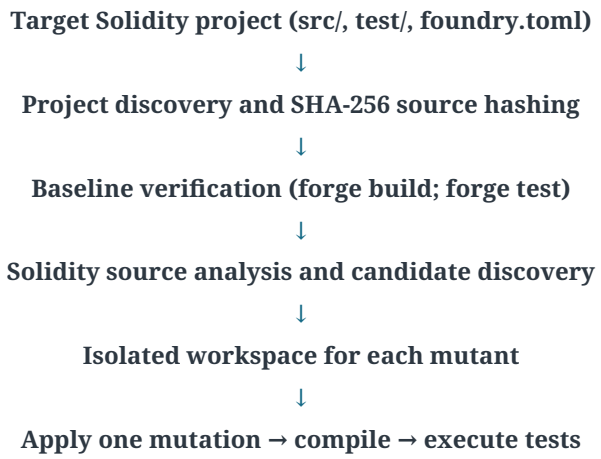
The initial implementation focuses on local Foundry projects. It assumes that the target project has Solidity source files, a Foundry configuration, and an existing test suite. The framework validates the baseline before beginning a mutation campaign and is designed not to overwrite the original source files.

The three operators are intentionally scoped to supported code patterns. They are not general-purpose vulnerability proofs. For example, timestamp use is not automatically a vulnerability, and moving an external call before a state update only constitutes a meaningful reentrancy mutation when the relevant contract logic and invariants support that interpretation.

- In scope: local source analysis, mutation generation, compilation, test execution, result classification, scoring, and report generation.
- Out of scope for the initial version: public-chain deployment, real-fund interaction, exhaustive formal verification, and universal support for every Solidity coding pattern.
- Research limitation: the mutation score reflects only the selected operators and eligible mutants; it does not quantify overall security.

3. System Architecture

The framework uses Python to orchestrate project discovery, source analysis, mutation generation, subprocess execution, classification, scoring, and reporting. Foundry/Forge provides Solidity compilation and test execution. Compiler/AST tooling supports identification of source locations and relevant constructs.



↓

Classify outcome and retain logs

↓

Calculate scores and verify original-source integrity

↓

JSON results + standalone HTML report

3.1 Core modules

Module	Responsibility
Project loader	Validates the project, discovers source files, and records source hashes.
Solidity parser/analyzer	Identifies relevant expressions, statements, source offsets, and mutation candidates.
Mutation engine	Applies one controlled transformation per mutant and records metadata.
Compiler manager	Invokes the appropriate Solidity/Foundry compilation process.
Test runner	Runs the original test suite against each mutant and captures output, exit codes, and duration.
Result classifier	Distinguishes killed, survived, invalid, equivalent, timeout, and unresolved outcomes.
Scoring engine	Calculates overall and operator-specific mutation metrics.
Report generator	Produces JSON and standalone HTML results.

3.2 Isolation and reproducibility

Each mutant is evaluated in a disposable workspace. The supplied implementation report states that SHA-256 hashes are used to verify that original source files remain unchanged. A single mutation is applied per experiment so that results can be attributed to a specific operator and source location. Execution logs, source hashes, and transformation metadata support reproducibility.

4. Mutation Operator Design

4.1 Unchecked arithmetic (UNCHECKED_MATH)

In Solidity 0.8 and later, arithmetic overflow and underflow normally trigger a revert. An unchecked block disables those checks for operations inside it. The operator identifies eligible arithmetic expressions and can wrap a supported expression in an unchecked block.

```
Original:
balances[msg.sender] += msg.value;

Mutant:
```

```
unchecked { balances[msg.sender] += msg.value; }
```

Candidate detection should reject operations already within an unchecked scope and should avoid mutations that are syntactically invalid or behaviorally irrelevant. The supplied campaign recorded six unchecked-math mutants, all of which survived. The report attributes this result to tests using small values that do not approach uint256 boundaries. This is a useful test-suite finding, but each survivor should still be reviewed to determine whether its mutation can alter observable behavior.

4.2 Reentrancy (REENTRANCY)

The Checks-Effects-Interactions pattern recommends updating relevant state before transferring control to an external contract. If a withdrawal function performs an external call before deducting the user's balance, a malicious recipient may re-enter while the old balance remains available.

```
Original ordering:
balances[msg.sender] -= amount;
(bool success, ) = payable(msg.sender).call{value: amount}("");
require(success, "Transfer failed");

Mutated ordering:
(bool success, ) = payable(msg.sender).call{value: amount}("");
require(success, "Transfer failed");
balances[msg.sender] -= amount;
```

This operator must be context-aware. It should only reorder statements when their scope and dependencies permit a meaningful transformation. The sample includes a recipient/attacker contract and a test that checks the state visible during the callback. The supplied results record one reentrancy mutant and one killed mutant, giving a reported operator score of 100% for that single candidate.

4.3 Timestamp reliance (TIMESTAMP_RELIANCE)

Timestamp-dependent code often implements deadlines, auctions, timelocks, or eligibility windows. The operator identifies comparisons involving block.timestamp and applies controlled boundary mutations, such as changing a strict comparison to a non-strict comparison.

```
Original:
require(block.timestamp < deadline, "Deadline passed");

Mutant:
require(block.timestamp <= deadline, "Deadline passed");
```

A boundary mutation is not automatically an exploitable vulnerability. Its purpose in mutation testing is to determine whether tests distinguish the original behavior from the altered behavior at relevant timestamps. The supplied results record four timestamp mutants, two killed and two surviving. The report recommends tests at the exact deadline and at timelock release boundaries.

5. Execution Workflow

- Validate the input path and detect the supported Foundry project structure.
- Run baseline compilation and tests; halt the campaign if the baseline is invalid or failing.
- Parse the source and enumerate candidates for enabled operators.
- Create a clean, isolated copy for each mutant and apply one transformation.
- Compile the mutant. Record compiler failures as invalid rather than automatically treating them as killed.
- If compilation succeeds, run the configured tests with a timeout and capture logs.
- Classify the outcome based on the available evidence, including attribution of test assertion failures.
- Calculate overall and per-operator metrics, retaining diagnostics for timeouts and unresolved runs.
- Generate JSON and HTML reports and verify that original source hashes remain unchanged.

5.1 Outcome classification

Status	Meaning
KILLED	A relevant test detects a meaningful behavior change introduced by the mutant.
SURVIVED	The test suite completes without detecting the intended behavior change.
INVALID	The mutant cannot compile or is structurally unusable.
EQUIVALENT	The mutant has no observable behavioral difference under the relevant specification; equivalence should not be assumed without evidence.
TIMEOUT	Compilation or test execution exceeds the configured time limit.
UNRESOLVED	Available evidence is insufficient to attribute the result reliably.

A nonzero test-runner exit code is not sufficient evidence of a killed mutant. The framework should distinguish assertion failures from compiler errors, setup failures, dependency problems, crashes, and timeouts. If attribution is ambiguous, the outcome should remain unresolved.

6. Mutation Score and Metrics

The standard mutation score measures the proportion of valid, non-equivalent mutants that are killed by the test suite.

$$\text{Mutation Score} = K / (M - I - E) \times 100\%$$

- M: total generated mutants.
- K: killed mutants.
- I: invalid mutants.

- E: equivalent mutants.

The supplied implementation report states that timeouts and unresolved executions remain visible and are not silently excluded from the denominator. This conservative policy prevents execution problems from artificially inflating the reported score. A zero or non-positive denominator must be handled safely and documented.

6.1 Additional metrics

- Operator-specific mutation score.
- Compilation success rate.
- Counts of killed, survived, invalid, equivalent/suspected-equivalent, timed-out, and unresolved mutants.
- Average and total execution time.
- Number of candidate sites discovered and rejected, with rejection reasons.
- Survivor analysis and recommended additional tests.

Mutation score is not a percentage measure of smart contract security. It is conditional on the operators, candidate-selection rules, generated mutants, and the quality of the test-oracle attribution.

7. Experimental Setup and Results

7.1 Benchmark contract

The supplied report describes a local Foundry example called Bank.sol, structured as a decentralized vault. It includes deposits, withdrawals, balance accounting, timelock behavior, and a deadline-based bonus operation. The withdrawal path follows Checks-Effects-Interactions in the original contract.

7.2 Test suites

Suite	Reported tests	Purpose
BankBaselineTest	6	Normal deposits and withdrawals, insufficient-balance behavior, timelock progression, and deadline behavior.
ReentrancyInvariantTest	1	Uses a recipient/attacker contract to inspect state during an external callback.
BankImprovedTest	2	Tests exact deadline boundary behavior and arithmetic boundary behavior.
Total	9	All reported sample Foundry tests passed.

7.3 Reported mutation campaign

Metric	Reported result
Target project	bank_project
Total mutants generated	11
Killed mutants	3
Surviving mutants	8
Invalid / compile errors	0
Equivalent / suspected	0
Timeouts	0
Unresolved	0
Compilation rate	100.0%
Total execution time	0.92 seconds
Overall mutation score	27.27%

The reported overall score is 3 / 11 × 100 = 27.27%, given the reported zero invalid and equivalent mutants. The interpretation depends on the validity and non-equivalence of the generated mutants and on accurate attribution of killed outcomes.

7.4 Operator-level results

Operator	Killed	Survived	Reported score
UNCHECKED_MATH	0	6	0.00%
REENTRANCY	1	0	100.00%
TIMESTAMP_RELIANCE	2	2	50.00%
Total	3	8	27.27%

These figures are reproduced from the supplied project report and are not independently re-executed in this document. They should be treated as the project's reported experimental results.

8. Survivor Analysis and Test Hardening

8.1 Unchecked arithmetic

All six reported arithmetic mutants survived. The supplied analysis indicates that the baseline tests use small values, so checked and unchecked arithmetic behave identically for the exercised inputs. Recommended follow-up: add boundary-focused fuzz tests and tests near the maximum representable uint256 value, while ensuring the expected semantics are explicit.

8.2 Reentrancy

The one reported reentrancy mutant was killed. The attacker recipient observed a nonzero internal balance during the callback, causing the invariant assertion to fail. This demonstrates that the sample suite can detect the specific reentrancy-related mutation tested. Because the sample contains only one such mutant, the 100% score should not be generalized to reentrancy faults as a whole.

8.3 Timestamp reliance

Two of four timestamp mutants survived. The supplied analysis says the exact-boundary mutation was killed by a test that warped time to the deadline, while other mutations survived because tests exercised times well away from the relevant boundary. Recommended follow-up: test immediately before, exactly at, and immediately after each deadline or release timestamp.

8.4 Before-and-after experiment

A useful next experiment is to add targeted tests for selected survivors and rerun the campaign. Report the change in score, the number of newly killed mutants, and any additional execution cost. Do not invent post-improvement results; record them only after actually running the improved suite.

9. Verification and Quality Assurance

9.1 Python tests

The supplied report records 24 pytest tests passing in 4.39 seconds, with a reported 100% pass rate across the tool's unit and integration tests. The listed tests cover project discovery, parsing, mutation operators, classification, scoring, report generation, and end-to-end execution.

9.2 Foundry tests

The supplied report records three Foundry test suites passing: six baseline tests, one reentrancy-related test, and two improved/boundary tests, for a total of nine passing tests and zero failures.

9.3 Source integrity

The implementation report describes SHA-256 source hashing before and after the campaign to verify that the original files were not modified. Each mutant is processed in an isolated temporary workspace, reducing the risk of one transformation affecting subsequent runs.

These test outcomes are reported from the supplied project notes. This report does not independently execute the repository or validate the logs; a final submission should retain the raw command output and report files as supporting evidence.

10. Deliverables

Deliverable	Purpose
CLI entry point (main.py)	Starts mutation campaigns and exposes command-line options.

Mutation engine	Coordinates candidate discovery, transformation, execution, and result collection.
Three operator modules	Implement unchecked arithmetic, reentrancy, and timestamp-related mutations.
Project loader and parser	Validate the project and locate eligible mutation sites.
Compiler manager and test runner	Compile mutants and execute tests in isolated workspaces.
Classifier and scoring module	Assign outcomes and calculate metrics.
JSON report	Provides machine-readable campaign results.
HTML report	Provides a standalone interactive summary and detailed mutant results.
Example Foundry project	Provides a reproducible benchmark contract and tests.
Documentation	Describes setup, architecture, operators, methodology, and limitations.

The supplied report lists these deliverables as being located in the project repository. File paths in that report refer to the author's local Windows environment and are not portable download links.

11. Limitations and Threats to Validity

- Operator coverage is limited to the three implemented fault classes and the patterns recognized by the candidate detector.
- Some mutations may be equivalent or may not produce a meaningful security effect; equivalence is difficult to establish automatically.
- Test failure attribution can be ambiguous when a mutation triggers setup errors or unrelated failures.
- Mutation scores depend on candidate selection, operator design, and the quality of the test suite.
- Compiler versions and project dependencies can affect whether mutants compile and execute.
- A high score does not prove the absence of vulnerabilities; a low score does not identify every missing security property.
- The reported sample has only 11 mutants, so its results demonstrate the workflow but are not a broad benchmark of Solidity testing practice.
- The supplied results have not been independently reproduced while preparing this report.

Future work may include more contract benchmarks, additional fault operators for access control and signature validation, improved AST-level transformations, better equivalence analysis, parallel campaign execution, and comparison with other mutation-testing tools.

12. Conclusion

This project applies mutation testing to Solidity smart contract test suites by injecting controlled faults related to unchecked arithmetic, reentrancy, and timestamp-dependent behavior. Its workflow combines project validation, source analysis, isolated mutation generation, Foundry compilation and test execution, outcome classification, mutation scoring, and multi-format reporting.

The supplied experimental results report an overall mutation score of 27.27% on the sample bank project. The operator breakdown identifies arithmetic-boundary testing as the most obvious weakness in the evaluated suite, while the tested reentrancy mutant was detected and half of the timestamp mutants were killed. These results illustrate how mutation testing can reveal gaps that conventional execution coverage may not expose.

The principal contribution is a reproducible way to evaluate test sensitivity to selected security-relevant faults. The framework should be presented as a test-quality assessment tool, not as a guarantee of contract security. Broader evaluation and independent reproduction would strengthen the conclusions.

Appendix A. Example Commands

```
python main.py --project ./examples/bank_project
python main.py --project ./examples/bank_project --operators unchecked_math
python main.py --project ./examples/bank_project --operators reentrancy,timestamp
python main.py --project ./examples/bank_project --timeout 120
python main.py --project ./examples/bank_project --output ./reports/run_001
python main.py --list-operators
```

These commands reflect the intended interface described in the supplied project report. Confirm exact option names against the implemented CLI before including them in a live demonstration.

Appendix B. References and Further Reading

The project notes identify Foundry/Forge as the execution framework and mention prior Solidity mutation-testing systems including SuMo, Vertigo, and Deviant. The following official or project pages are useful starting points; consult the original publications for formal citation details.

- Foundry documentation: https://getfoundry.sh/
- Foundry invariant testing: https://getfoundry.sh/forge/invariant-testing
- SuMo Solidity mutation tool: https://github.com/MorenaBarboni/SuMo-SOlidity-MUtator

A final academic submission should add complete bibliographic entries for any papers actually consulted, including authors, title, venue, year, and DOI or stable URL where available.